

RAPPORT DE PROJET

Phases 1 & 2

Projet : La Table des Jedi / Creative Yumland

Filière : préING2 — 2025-2026

Groupe : Leroy Yannis · Abderrahmane Mehdi · Moudakkir Ilyes

■ Phase 1 — Identité visuelle & Structure HTML/CSS

1. Planning et Répartition Individuelle des Tâches

La Phase 1 a été consacrée à la création de l'identité visuelle et à la mise en place de la structure de base du site en HTML et CSS. Afin de permettre une évaluation individuelle, voici la répartition précise des contributions :

Membre	Part	Contributions principales
Leroy Yannis	33 %	Conception et intégration des pages métiers spécifiques (interfaces Livreur et Cuisine/Restaurateur), développement du système de filtres de produits en CSS pur.
Abderrahmane Mehdi	33 %	Création de la charte graphique globale (fichier common.css), développement de la page d'accueil et optimisation du responsive design pour tous les écrans.
Moudakkir Ilyes	34 %	Création des formulaires d'authentification (inscription/connexion), développement de la console d'administration et rédaction de la documentation technique finale.

L'ensemble du groupe a participé à la définition commune du thème « Star Wars » et au nommage du restaurant.

2. Problèmes Rencontrés

Difficultés techniques

- Gestion du slider : difficulté à synchroniser les flèches de navigation avec l'affichage des images en utilisant uniquement du CSS.
- Fichiers et ressources : problèmes d'affichage de certaines polices spéciales et ralentissement dû au poids de la vidéo d'arrière-plan.
- Mise en page : superposition d'éléments sur les petits écrans, notamment au niveau du menu de navigation.

Difficultés d'organisation

- Structure du code : accumulation de styles rendant les modifications complexes à cause de règles qui s'annulaient.

- Gestion du temps : temps important passé sur les détails visuels (effets néon), ralentissant la finalisation des pages statiques.

Difficultés humaines

- Choix artistiques : discussions pour fixer le niveau d'immersion visuelle du site (sobre vs totalement immersif).

3. Solutions Apportées

Domaine	Solution mise en place
Technique	Utilisation de boutons invisibles (cases à cocher) en CSS pour changer les images du slider.
Technique	Réorganisation complète du fichier CSS pour bien séparer les styles de chaque section.
Technique	Ajout de filtres de flou sur l'arrière-plan pour maintenir la lisibilité du texte sur la vidéo.
Organisationnel	Création d'une structure de dossiers normalisée pour les images et icônes.
Ergonomie / Technique	Création d'un bouton de confirmation de livraison sous forme de « Slider » (glissière) en CSS pur pour le livreur, afin d'éviter les fausses manipulations avec des gants.
Organisationnel	Vérification systématique de l'affichage sur navigateur après chaque changement important.

4. Conclusion de la Phase 1

Les objectifs visuels ont été remplis. Le site dispose d'une identité forte et d'une base solide pour intégrer la logique serveur (PHP) et le dynamisme (JavaScript) lors des prochaines phases.

■ Phase 2 — Dynamisme PHP, API & Expérience Utilisateur

1. Planning et Répartition Individuelle des Tâches

La Phase 2 a porté sur l'intégration du backend PHP, du flux de commande complet, de l'API de paiement CyBank et de la charte graphique sci-fi. Les tâches ont été réparties pour que chacun prenne en charge un périmètre fonctionnel cohérent.

Membre	Part	Contributions principales
Abderrahmane Mehdi	33 %	Mise en place de l'architecture PHP : structure des fichiers, includes, routage entre les pages. Développement du panier (panier.php) avec gestion des quantités en session, page de validation de commande et traitement initial (traitement_commande.php) qui génère les entrées dans commandes.json.
Leroy Yannis	34 %	Intégration de l'API CyBank : construction de la requête signée (initier_paiement.php), redirection vers la plateforme externe, vérification du hash de retour (retour_paiement.php) et mise à jour du statut. Développement de la page profil dynamique (profil.php) avec historique des commandes et système de notation par étoiles (notation.php) pour les commandes livrées.

Membre	Part	Contributions principales
Moudakkir Ilyes	33 %	Tableau de bord restaurateur (commandes.php) : affichage des commandes en attente et en livraison, distinction des commandes planifiées, boutons de changement de statut. Interface livreur mobile (livraison.php) avec confirmation ou abandon de livraison. Corrections UX/UI : champ adresse conditionnel selon le mode de retrait, boutons de déconnexion sur les pages métier, application de la charte sci-fi sur toutes les pages. Création des données de test dans users.json.

2. Choix du Format de Stockage des Données

Pour stocker les données du site, nous avons choisi le format **JSON** plutôt que le CSV. Voici pourquoi ce choix nous a semblé plus adapté :

- **Structure imbriquée** : une commande contient une liste de plats, chacun avec son nom, sa quantité et son prix. En CSV, il aurait fallu une table séparée ou imbriquer des données dans une cellule, ce qui complique la lecture. En JSON, cette hiérarchie s'exprime naturellement avec un tableau d'objets.
- **Intégration PHP directe** : PHP lit et écrit le JSON en une ligne avec `json_decode()` et `json_encode()`. Pas besoin de parser manuellement les lignes comme avec `fgetcsv()`.
- **Types de données conservés** : JSON distingue les entiers, les décimaux, les chaînes et les valeurs nulles. Cela évite d'avoir à convertir manuellement chaque champ comme en CSV où tout est du texte.
- **Facilité d'évolution** : quand nous avons eu besoin d'ajouter de nouveaux champs (etage, code_interphone, montant_paye, mode_retrait), il a suffi de les ajouter aux nouvelles entrées. Les anciennes restent lisibles sans modifier toute la structure.

Nous avons quatre fichiers JSON dans le dossier **data/**, séparé des scripts PHP :

Fichier	Contenu	Modifié par
users.json	Comptes utilisateurs (tous rôles)	inscription.php, connexion.php
commandes.json	Historique et état des commandes	retour_paiement.php, livraison.php, commandes.php
plats.json	Catalogue des 15 plats	Lecture seule
menus.json	3 menus préconfigurés	Lecture seule

L'intégrité des écritures simultanées dans commandes.json est protégée par un verrouillage de fichier PHP (flock) dans la fonction write_json() pour éviter les conflits entre le restaurateur et le livreur.

3. Réalisations Techniques

Architecture du projet

Les fichiers PHP sont organisés en sous-dossiers dans pages/ : les pages accessibles par l'utilisateur sont à la racine de pages/, les fichiers partagés (header.php, footer.php, fonctions.php, getapikey.php) sont dans pages/includes/, et les données JSON sont dans data/. Les feuilles de style sont dans css/ et les images dans images/.

Abderrahmane Mehdi — Panier et traitement de commande

- panier.php : récupération des plats depuis plats.json, calcul du total depuis la session PHP, affichage du tableau avec boutons pour retirer un article (retirer_panier.php). Formulaire de validation avec choix du type (immédiate ou planifiée), du mode de retrait (livraison ou à emporter), de l'heure cible et de l'adresse.

- `ajouter_panier.php` / `retirer_panier.php` : gestion de la session panier, incrémentation et décrémentation des quantités.
- `initier_paiement.php` : calcul du montant final depuis `plats.json`, génération d'un identifiant de transaction unique, sauvegarde de la commande en session pour survivre au retour CyBank, construction du hash MD5 de contrôle et redirection automatique vers la plateforme CyBank.

Leroy Yannis — API CyBank, profil et notation

- `retour_paiement.php` : récupération des paramètres GET renvoyés par CyBank, recalcul du hash attendu et comparaison avec le hash reçu. Si le paiement est accepté et le hash valide, la commande est écrite dans `commandes.json` avec le statut 'en attente' et le `transaction_id`. Le panier est ensuite vidé de la session.
- `profil.php` : lecture de `users.json` pour afficher les informations de l'utilisateur connecté (nom, adresse, date d'inscription, dernière connexion). Calcul du niveau de fidélité (Padawan, Chevalier Jedi, Maître Jedi, Grand Maître) selon les points. Affichage de l'historique des commandes du client avec un bouton 'Noter' pour les commandes au statut livré.
- `notation.php` : page accessible via un lien depuis `profil.php` avec l'id de la commande en paramètre GET. Vérification que la commande appartient bien au client connecté et qu'elle est au statut livré. Formulaire avec deux séries d'étoiles (qualité des plats et livraison) en CSS pur sans JavaScript, plus un commentaire libre. Les notes sont enregistrées dans `commandes.json`.

Moudakkir Ilyes — Tableau de bord restaurateur, interface livreur et finitions

- `commandes.php` : page réservée au restaurateur, affiche les commandes en attente dans une colonne gauche et les commandes en livraison ou terminées (5 dernières) dans une colonne droite. Les commandes planifiées sont identifiées visuellement avec leur heure cible (champ `heure_livraison`). Boutons pour passer une commande en livraison, la marquer comme livrée ou abandonnée. Un select de livreur est présent dans l'interface mais l'attribution manuelle effective est prévue pour la Phase 3.
- `livraison.php` : interface pensée pour mobile, affiche la commande assignée au livreur connecté (statut 'en livraison' avec son `id_livreur`). Affichage de l'adresse avec un lien vers Google Maps, détail des plats et total. Boutons de confirmation de livraison ou de signalement d'abandon, qui écrivent dans `commandes.json` avec flock.
- Corrections UX : le champ adresse est masqué en mode 'À emporter' grâce à du JavaScript léger. Boutons de déconnexion clairs sur toutes les pages métier (admin, restaurateur, livreur). Charte graphique sci-fi appliquée uniformément sur toutes les pages.
- Données de test : 7 clients, 1 livreur (Luke Skywalker), 1 restaurateur (Dexter Jettster), 2 administrateurs (Palpatine, Dark Vador) dans `users.json`. 15 plats dans `plats.json`, 3 menus dans `menus.json`.

4. Problèmes Rencontrés et Solutions Apportées

Domaine	Problème	Solution mise en place
Technique	Écritures simultanées dans <code>commandes.json</code> entre le restaurateur et le livreur pouvaient corrompre le fichier.	Ajout d'un verrou de fichier PHP (flock) dans <code>write_json()</code> pour qu'une seule écriture ait lieu à la fois.
Technique	Le format du hash de contrôle CyBank ne correspondait pas à la documentation : le statut retourné s'appelle 'status' et non 'statut' dans l'URL de retour.	Inspection des paramètres GET reçus et ajustement de l'ordre des champs dans le calcul MD5 côté <code>retour_paiement.php</code> .

Domaine	Problème	Solution mise en place
Technique	Le champ adresse devait s'afficher ou se masquer selon le mode de retrait choisi par le client.	Adresse toujours affichée
Organisationnel	Risque de conflits Git sur les fichiers partagés comme commandes.json et header.php.	Convention de nommage des branches et relecture croisée avant chaque fusion.
UX / Design	Pages développées séparément avec des styles légèrement différents.	Session de relecture commune en fin de phase pour harmoniser les classes CSS.

5. Vérification des Fonctionnalités Phase 2

Tableau récapitulatif des fonctionnalités demandées dans le cahier des charges et de leur état dans notre projet :

Fonctionnalité demandée	Statut	Détail
Inscription fonctionnelle	✓	inscription.php — ajout dans users.json avec rôle client par défaut.
Connexion fonctionnelle	✓	connexion.php — vérification login/mot de passe, création de session, redirection selon le rôle, mise à jour de dernière_connexion.
Vérification compte banni	✓	connexion.php — si statut = 'banni', message d'erreur, accès refusé.
Admin : liste des utilisateurs avec affichage des boutons d'action	✓	admin.php — tableau dynamique depuis users.json avec badges rôle/statut et boutons voir/éditer/bannir (actions effectives en Phase 3).
Restaurateur : liste des commandes	✓	commandes.php — commandes en attente et en livraison, distinction commandes planifiées avec heure cible affichée.
Restaurateur : changer le statut d'une commande	✓	commandes.php POST — passage en livraison, livré ou abandonné via des boutons. Attribution automatique du premier livreur actif.
Attribution manuelle d'un livreur par le restaurateur	Phase 3	Le select est présent dans l'interface mais l'action n'est pas encore effective, conformément au cahier des charges.
Livreur : voir sa commande et confirmer la livraison	✓	livraison.php — affiche la commande assignée (statut 'en livraison'), boutons confirmer livré ou signaler abandon, écriture dans commandes.json.
Client : historique des commandes	✓	profil.php — affiche toutes les commandes du client connecté, les plus récentes en premier.
Client : noter une commande livrée	✓	notation.php — accessible depuis profil.php, deux notes par étoiles (qualité et livraison) + commentaire, enregistré dans commandes.json.
Client : page de profil	✓	profil.php — informations personnelles, niveau de fidélité avec barre de progression, avantages débloqués. Modification effective en Phase 3.
Client : panier + paiement CyBank	✓	panier.php → initier_paiement.php → CyBank → retour_paiement.php. Vérification du hash, écriture de la commande si accepté.

Fonctionnalité demandée	Statut	Détail
Commandes immédiates et planifiées	✓	Champ type = immediate ou planifiée avec heure_livraison. Le restaurateur voit les commandes différées séparément.
≥5 clients, ≥2 admins, ≥3 menus, ≥15 plats en données de test	✓	7 clients + 2 admins + 1 livreur + 1 restaurateur dans users.json. 15 plats dans plats.json. 3 menus dans menus.json.
Arborescence PHP propre	✓	pages/ pour les vues, pages/includes/ pour les fichiers partagés, data/ pour les JSON, css/ pour les styles.

6. Conclusion de la Phase 2

La Phase 2 marque l'aboutissement fonctionnel du projet. Le flux de commande est complet de bout en bout : le client ajoute des plats au panier, choisit un mode de livraison et une heure, paie via CyBank, et peut ensuite suivre sa commande depuis son profil et la noter une fois livrée. Le restaurateur gère les commandes depuis son tableau de bord et le livreur confirme la livraison depuis une interface pensée pour mobile.

Le stockage JSON s'est avéré bien adapté à notre projet. La flexibilité du format nous a permis d'ajouter des champs au fur et à mesure (etage, code_interphone, montant_paye) sans tout réécrire. Les quatre fichiers couvrent l'ensemble des données manipulées par le site.

La prochaine étape est la Phase 3 (JavaScript et requêtes asynchrones) qui permettra notamment de rendre les modifications de profil effectives et d'améliorer l'attribution des livreurs par le restaurateur.

■ Phase 3 — JavaScript & Requêtes Asynchrones

1. Planning et Répartition Individuelle des Tâches

La Phase 3 a porté sur l'intégration du JavaScript côté client et des requêtes asynchrones (fetch API) pour rendre le site interactif sans rechargement de page. La répartition suit la continuité fonctionnelle des phases précédentes.

Membre	Part	Contributions principales
Leroy Yannis	34 %	Changement de thème clair/sombre : bouton dans header.php, chargement/déchargement de css/light-theme.css sans rechargement, persistance via cookie theme (max-age 1 an), lecture PHP côté serveur. Compteurs de caractères temps réel (pseudo, email, mot de passe) avec alerte orange <=5 caractères restants. Toggle affichage/masquage mot de passe sur inscription.php et connexion.php.
Abderrahmane Mehdi	33 %	Validation JS complète des formulaires : règles mdp en temps réel avec retour OK/KO par critère (inscription.php), longueur identifiant et présence mot de passe (connexion.php), e.preventDefault() dans tous les cas. Modification du profil en connexion asynchrone (profil.php) : bouton crayon par champ, fetch() -> api/update_profil.php, DOM mis à jour. Modification de commande avant préparation (modifier_commande.php) : total automatique, paiement complémentaire CYBank si surplus, bon de réduction si déficit.
Moudakkir Ilyes	33 %	Tableau de bord restaurateur asynchrone (commandes.php) : boutons PRÉPARER/EN LIVRAISON/ABANDONNER via fetch(), déplacement dynamique des cartes, sélection manuelle du livreur. Filtres et tris produits (produits.php) : fetch('./api/get_products.php') avec loader, debounce 300 ms, tri local applyLocalSort(). Bannissement asynchrone admin.php, coupure de session immédiate, détection contexte api/ dans fonctions.php, modale modification rôle/statut. Confirmation livraison et notation async.

L'ensemble du groupe a contribué aux tests croisés et à l'harmonisation des comportements asynchrones sur toutes les pages métier.

2. Réalisations Techniques

Changement de thème clair/sombre

Un bouton soleil/lune ajouté dans **header.php** modifie dynamiquement l'attribut **href** du **<link id="theme-link">** pour charger ou décharger **css/light-theme.css** sans rechargement. Le choix est persisté dans un cookie **theme** (max-age : 1 an) ; PHP lit ce cookie au chargement et applique le bon fichier CSS côté serveur. Le thème clair désactive la vidéo de fond et les effets lumineux (fond #eef2f5, texte #1f2937, accent bleu #0ea5e9).

Compteurs de caractères et toggle mot de passe

- Compteurs temps réel sur **inscription.php** (pseudo 0/30, email 0/50, mdp 0/50) et **connexion.php** (identifiant 0/30, mdp 0/50). Le compteur vire en orange #ff8844 quand il reste <=5 caractères.
- Bouton oeil sur **inscription.php** et **connexion.php** : bascule type="password" / type="text", icône affichage/masquage.

Validation côté client des formulaires

- **inscription.php** : pseudo >=3 caractères, regex email, règles mot de passe (8 chars min, majuscule, chiffre, caractère spécial) vérifiées en temps réel avec retour OK/KO par règle.
- **connexion.php** : longueur identifiant minimum, présence mot de passe.
- **notation.php** : vérification que les deux critères sont notés avant soumission — message d'erreur inline sans rechargement.
- Dans tous les cas : **e.preventDefault()** bloque l'envoi HTTP si erreur.

Modification du profil en connexion asynchrone — profil.php

Un bouton crayon par champ (login, prénom, nom, email, téléphone, adresse) remplace le span par un input. Après validation client, un **fetch()** envoie un objet JSON à **api/update_profil.php** (POST) et met à jour le DOM sans rechargement. Les erreurs serveur (double email, etc.) s'affichent en ligne sous le champ.

Modification de commande avant préparation — modifier_commande.php

Accessible depuis **profil.php** pour les commandes au statut « en attente ». Ajout/retrait de plats et menus avec total mis à jour automatiquement côté client. Si le nouveau total dépasse l'initial, un paiement complémentaire est initié via **initier_paiement_modif.php** (CYBank) ; dans le cas contraire, un bon de réduction est généré en crédit. L'envoi final passe par **fetch('../api/modifier_commande.php')** (POST JSON).

Filtres et tris sur la page produits — produits.php

- Filtres catégorie et régime alimentaire -> **fetch('../api/get_products.php')** avec loader animé.
- Recherche texte avec **debounce 300 ms** -> fetch asynchrone.
- Tri prix croissant/décroissant -> tri local sur le DOM via **applyLocalSort()** sans appel réseau, exécuté après chaque fetch.

Gestion des commandes par le restaurateur — commandes.php

- Tous les boutons d'action (PRÉPARER, PRÊTES, EN LIVRAISON, ABANDONNER) utilisent **fetch('../api/update_statut_commande.php')**.
- Mise à jour du DOM sans rechargement : badge statut et boutons suivants affichés progressivement. Quand une commande passe en livraison, sa carte est déplacée dynamiquement dans la colonne droite.
- Sélection manuelle du livreur via **<select>** embarqué dans la carte : **id_livreur** inclus dans la requête fetch.

Bannissement asynchrone et coupure de session — admin.php

- Bouton ban/unban -> **fetch('../api/toggle_ban.php')** -> mise à jour du badge statut et couleur de ligne sans rechargement.
- Dans **functions.php** : vérification via chemin absolu **__DIR__ . '/../data/users.json'**. Si statut = 'banni' : **session_unset()** + **session_destroy()** + redirection vers connexion.php?banni=1.
- Contexte **api/** détecté via **basename(dirname(\$_SERVER['PHP_SELF']))** : réponse JSON d'erreur au lieu d'une redirection HTML. Message orange affiché sur la page de connexion quand ?banni=1.
- Modification de rôle et statut premium via modale -> **fetch('../api/update_user.php')**.

Les boutons de confirmation de livraison (**livraison.php**) et de notation (**notation.php**) utilisent également fetch pour mettre à jour le DOM : bannière MISSION ACCOMPLIE/ANNULEE pour le livreur, blocage JS des doubles notations complétant la protection serveur existante.

3. Problèmes Rencontrés et Solutions Apportées

Domaine	Problème	Solution mise en place
---------	----------	------------------------

Technique	Le chemin relatif '../data/users.json' dans fonctions.php échouait silencieusement quand le fichier était inclus depuis api/.	Remplacement par __DIR__ . '../data/users.json' : chemin absolu depuis la position physique du fichier, indépendant du fichier appelant.
Technique	La redirection HTTP header('Location:') depuis un endpoint JSON cassait la réponse JSON lors du ban d'un utilisateur en cours de requête fetch.	Détection du contexte via <code>basename(dirname(\$_SERVER['PHP_SELF']))</code> : retour d'une réponse JSON {success: false} depuis api/ au lieu d'une redirection HTML.
Technique	Filtres et tri déclenchaient deux mécanismes différents (local vs async) pouvant se chevaucher.	Séparation nette : filtres -> fetch reconstruit les cartes ; tri -> <code>applyLocalSort()</code> opère sur le DOM existant, appelée après chaque fetch.
Technique	Les nouveaux boutons d'action créés dynamiquement (déplacement d'une commande vers la colonne livraison) n'avaient pas d'écouteur d'événement.	Attachement systématique de window.actionClickHandler sur tous les nouveaux boutons .js-action-cmd après injection dans le DOM.

4. Vérification des Fonctionnalités Phase 3

Tableau récapitulatif des fonctionnalités demandées dans le cahier des charges et de leur état dans notre projet :

Fonctionnalité demandée	Statut	Détail
Changement de thème clair/sombre via bouton	OK	header.php + light-theme.css chargé sans rechargement.
Persistence du thème via cookie	OK	Cookie theme (1 an), lu par PHP au chargement.
Toggle affichage mot de passe	OK	inscription.php et connexion.php, icône oeil.
Compteurs de caractères temps réel	OK	Tous les champs limités sur inscription et connexion.
Validation JS inscription	OK	Pseudo, email regex, règles mdp en temps réel.
Validation JS connexion	OK	Identifiant >=3 chars, mot de passe non vide.
Validation JS notation	OK	Deux critères obligatoires avant soumission.
Profil : modification async champ par champ	OK	fetch() -> api/update_profil.php, DOM mis à jour.
Produits : filtres asynchrones	OK	fetch('../api/get_products.php') avec loader animé.
Produits : tri local	OK	Prix croissant/décroissant sur le DOM existant.
Modification commande avant préparation	OK	Total auto, paiement différence CYBank si surplus.
Restaurateur : changement statut async	OK	fetch('../api/update_statut_commande.php').
Restaurateur : assignation manuelle livreur	OK	Select embarqué + id_livreur dans la requête.
Admin : ban/unban async	OK	fetch('../api/toggle_ban.php'), DOM mis à jour.
Admin : session banni coupée immédiatement	OK	fonctions.php relit le JSON à chaque requête.
Admin : modification rôle/statut premium async	OK	Modale + fetch('../api/update_user.php').

Livreur : confirmer livraison async	OK	fetch() -> api/update_statut_commande.php.
Notation une seule fois par commande livrée	OK	Vérification serveur + validation JS.

5. Conclusion de la Phase 3

La Phase 3 marque la transition d'un site web statiquement servi vers une véritable application interactive. L'ensemble des pages métier bénéficie désormais de retours visuels immédiats : validation en ligne des formulaires, mise à jour du DOM sans rechargement pour le restaurateur, le livreur et l'administrateur, et filtrage asynchrone du catalogue produits.

L'architecture PHP-JSON héritée de la Phase 2 s'est révélée bien adaptée à l'ajout de la couche asynchrone : les endpoints **api/** répondent en JSON pur et sont consommés par fetch sans modifier la logique serveur existante. La détection de contexte dans `functions.php` garantit que le ban d'un utilisateur est immédiatement effectif quelle que soit la requête en cours.

La prochaine étape est la Phase 4 (soutenance et fonctionnalité innovante), au cours de laquelle le groupe présentera l'ensemble du projet et démontrera la fonctionnalité développée en autonomie.